INDIAN INSTITUTE OF TECHNOLOGY MADRAS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FINAL INTEGRATED ASSIGNMENT

FIRST-YEAR COMPUTER SCIENCE AND ENGINEERING

C Programming Language | Linux & Bash | Python Programming

ASSESSMENT PARTICULARS	DETAILS
Academic Level	End of First Year / Pre-Promotion Assessment
Applicable Subjects	C Programming Language; Linux and Bash; Python Programming
Maximum Marks	100
Duration / Submission Period	16 calendar days from date of issue
Assessment Mode	Individual practical assignment with documentation and viva voce
Programming Languages	C and Python
Operating Environment	Linux with Bash
Academic Session	2025 – 2026
Assignment Issue Date	30 / 10 / 2026
Submission Deadline	15 / 11 / 2026, 11:59 PM
Course / Paper Code	CS2500X, CS260L, CS260P
Assignment Reference Number	IITM / AAD / CSE / 132
Faculty / Evaluator Name	Shyam Iyer Reddy

STUDENT DETAILS	To be completed by student
Student Name	
Roll / Registration Number	
Programme / Section	
Email / Institutional ID	
Student Signature	

AUTHORIZED SIGNATURE	OFFICIAL SEAL / STAMP

DOCUMENT CONTROL AND CONTENTS

Document Item	Details
Document Title	Final Integrated Assignment — First-Year CSE
Prepared By	Department of Computer Science and Engineering
Reviewed By	Prof. Dr. Rajeev Raman
Approved By	Prof. Madhu Muthyam / HoD of CSE
Version	1.0
Issue Date	30 / 10 / 2026
Applicable Batch	CS25B / 2025
Document Classification	For enrolled students and authorized academic staff

Contents

1. Official Assignment Notice 3
2. Assignment Overview and Learning Outcomes 3
3. General Instructions and Submission Rules 3
4. Section A — C Programming Language (35 Marks) 5
5. Section B — Linux and Bash (25 Marks) 7
6. Section C — Python Programming (25 Marks) 9
7. Section D — Integrated Mini-Project (15 Marks) 10
8. Submission Structure and Verification Checklist 11
9. Assessment Rubric and Grading Guidelines 12
10. Academic Integrity and Technical Rules 12
11. Suggested 16-Day Work Plan 13
12. Viva Voce Question Bank 13
13. Student Declaration 14
14. Faculty Assessment Record 15
15. Final Evaluation and Approval 16



Head of Department
Computer Science and Engineering
Indian Institute of Technology Madras

1. Official Assignment Notice

This assignment is designed as a comprehensive end-of-year assessment for first-year Computer Science and Engineering students before progression to the second year. It evaluates theoretical understanding, practical programming ability, debugging, problem-solving, Linux command-line proficiency, and the ability to develop and document a small integrated software solution.

Students must demonstrate that they can write reliable programs, explain implementation decisions, work with files and processes, validate data, test edge cases, and use the Linux development environment effectively. The standard is intended to be rigorous but appropriate for the completion of first-year study.

2. Assignment Overview and Learning Outcomes

Component	Marks	Primary Competencies
C Programming	35	Pointers, dynamic memory, structures, linked lists, files, bitwise operations, debugging and algorithms
Linux and Bash	25	Filesystem, permissions, processes, pipelines, shell scripting, diagnostics and data processing
Python Programming	25	Functions, collections, validation, CSV processing, OOP, recursion and unit testing
Integrated Mini-Project	15	Cross-language workflow, reliability, testing, documentation and reproducibility
TOTAL	**100**	**Integrated first-year practical competence**

On completion, students should be able to:

- Design, implement, compile, test and debug modular C programs.
- Use pointers, dynamic memory allocation, structures, linked data structures and file operations safely.
- Navigate and manage files in Linux, interpret permissions, inspect processes and use command pipelines.
- Write Bash scripts that validate inputs, automate compilation and report failures through exit statuses and logs.
- Develop Python programs using functions, built-in data structures, classes, exceptions and the standard library.
- Create reproducible tests and explain algorithmic time and auxiliary-space complexity.
- Integrate C, Bash and Python components into a documented end-to-end workflow.

3. General Instructions and Submission Rules

1. This is an individual assignment. Each student must be able to explain and demonstrate the submitted work.
2. The submission period is 16 calendar days from the official issue date unless the department specifies otherwise.
3. Submit one compressed archive named **Final_Assignment_RollNumber.zip**, together with any separately requested declaration or cover sheet.
4. All source files must be readable and appropriately named. Include comments where they clarify non-obvious logic; avoid unnecessary comments that merely repeat the code.

5. Programs must validate input and handle expected errors rather than terminate unexpectedly or silently produce misleading output.
6. For C, compile with suitable warnings enabled. For Python, use Python 3 and the standard library unless additional dependencies are justified and documented.
7. Include genuine terminal output or screenshots as evidence. Do not fabricate test results or screenshots.
8. Use fictional records in all sample datasets. Do not include real student personal information, passwords, access tokens, private keys or other secrets.
9. External references may be consulted. Disclose substantial external code or AI assistance according to institutional policy.
10. The student must retain a backup of the submission until marks are finalized.
11. Late penalties, pass thresholds and promotion decisions are governed by the institution's published regulations, not by this template alone.

Recommended environment: a supported Linux distribution, Bash, GCC, and Python 3. Record the relevant tool versions in README.md. Students may use a local Linux installation, a virtual machine, or a permitted cloud development environment such as GitHub Codespaces, provided the submitted work can be reproduced.

4. Section A — C Programming Language (35 Marks)

Maximum Marks: 35

Answer all seven questions. Each question carries 5 marks.

Question 1. Pointer Arithmetic and Dynamic Memory Management [5 marks]

Write a C program that dynamically allocates memory for an array of n integers and performs the following operations:

(a) Read n integers from the user.

(b) Calculate the sum and arithmetic mean using pointer arithmetic for the calculations.

(c) Find the minimum and maximum values.

(d) Reverse the array in place using pointers.

(e) Release all dynamically allocated memory.

Requirements: Validate the input size, check allocation success, avoid memory leaks and invalid memory access, and explain the time complexity of reversal.

Question 2. Structures, Functions and Data Processing [5 marks]

Design a student-record management program using a C structure named Student. Each record must contain a roll number, student name, marks in three subjects, calculated total and percentage.

Implement functions to accept multiple records, calculate totals and percentages, sort students by total marks in descending order, search by roll number, and display the highest-scoring student.

Requirements: Use functions for individual operations. Handle duplicate roll numbers and invalid marks appropriately.

Question 3. Linked Lists and Dynamic Data Structures [5 marks]

Implement a singly linked list in C with insertion at the beginning and end, insertion at a specified position, deletion by value, searching, iterative reversal, display, and freeing of all remaining nodes.

Explain the difference between array allocation and linked-list allocation. State the time complexity of insertion at the head, searching, and reversal.

Requirements: Check allocation results and handle an empty list, an invalid position, and deletion of a value that does not exist.

Question 4. File Handling and Persistent Records [5 marks]

Develop a C program that maintains a product inventory. Each product contains an ID, name, quantity and unit price.

Support adding a product, listing stored products, searching by ID, updating quantity, calculating total inventory value, and handling a missing or unreadable data file.

Requirements: Records must persist between executions. Use appropriate file modes and check file-operation return values. Document the storage format.

Question 5. Bitwise Operations and Integer Representation [5 marks]

Write a C program that accepts an unsigned integer and: (a) displays its bits using a clearly defined bit width; (b) sets a selected bit; (c) clears a selected bit; (d) toggles a selected bit; (e) tests whether a selected bit is set; and (f) counts set bits.

Use bitwise operators and explain the masks used.

> **Requirements:** Validate bit positions and avoid undefined shifts. Clearly state whether positions are zero-based.

Question 6. Debugging, Undefined Behaviour and Code Analysis [5 marks]

> Analyse the following program. Identify its memory-safety defects, explain why the loop condition is incorrect, explain the effect of accessing memory after free(), rewrite the program safely, and describe how compiler warnings and a memory-safety diagnostic tool can help.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(3 * sizeof *p);

    if (p == NULL)
        return 1;

    for (int i = 0; i <= 3; i++)
        p[i] = i * 10;

    free(p);
    printf("%d\n", p[1]);

    return 0;
}
```

> Your explanation must distinguish intended behaviour from behaviour guaranteed by the C language.

Question 7. Algorithmic Problem Solving [5 marks]

> Write a C program that determines whether a given string is a palindrome, ignoring letter case and non-alphanumeric characters. For example, “A man, a plan, a canal: Panama!” should be classified as a palindrome.
>
> Use functions to separate input processing and palindrome checking. Handle empty input and input containing spaces. Do not use a third-party string-processing library. Explain the algorithm and state its time and auxiliary-space complexity.
>
> **Requirements:** Use a safe input strategy and ensure character classification is applied correctly.

5. Section B — Linux and Bash (25 Marks)

Maximum Marks: 25

Answer all five questions. Each question carries 5 marks.

Question 8. Filesystem Navigation and Permissions [5 marks]

> Using a Linux terminal, (a) display the current working directory; (b) create the directory structure below; (c) create a README file in the root; (d) copy a source file into the appropriate directory and rename the copy; (e) display file details, including permissions and sizes; and (f) set suitable permissions so the owner can read and write a file while other users have read-only access.

```
final_assignment/
├── c_programs/
├── python_programs/
├── scripts/
├── data/
└── reports/
```

> Explain absolute paths, relative paths, file permissions, and the meaning of r, w and x. Submit commands and genuine terminal evidence.

Question 9. Process Management and Redirection [5 marks]

> Demonstrate the following: (a) explain process ID and parent process ID; (b) list running processes and identify a process by name; (c) run a command in the background and inspect its status; (d) redirect standard output and standard error to separate files; (e) use a pipeline to filter output and count matching lines; and (f) explain >, >>, 2> and |.
>
> **Requirements:** Use harmless sample processes and files. Do not terminate system-critical processes.

Question 10. Bash Automation [5 marks]

> Write an executable Bash script named run_tests.sh that checks whether a specified source directory exists, finds all C source files in it, compiles each file separately using GCC with suitable warnings enabled, stores executables in a designated output directory, logs compilation success and failure, and returns a nonzero exit status if any compilation fails.
>
> **Requirements:** Quote variable expansions appropriately, handle filenames containing spaces, use meaningful exit codes, include a shebang and comments, and demonstrate the script with at least three C files, including one deliberate compilation error.

Question 11. Linux Diagnostic and Troubleshooting Report [5 marks]

> Prepare a short diagnostic report containing commands and observed output for: operating-system and kernel information; available disk space; memory usage; current user and group information; network interface and IP configuration; and a connectivity test to a reachable host where network access permits.
>
> For each command, explain the information it provides and identify one practical situation in which it would be useful.
>
> **Requirements:** Do not publish passwords, access tokens, private keys or other sensitive system information. If a check cannot be run, explain the limitation rather than inventing output.

Question 12. Shell-Based Data Processing [5 marks]

> Create a text file containing at least 20 fictional student records in a consistent format, such as:
>
> ```
> 101,Aarav,78
> 102,Diya,91
> 103,Kabir,64
> ```
>
> Write a Bash script that validates the input file, counts records, finds the highest and lowest marks, calculates the average, lists students scoring at least 75, and writes a formatted report to a separate output file.
>
> Handle malformed records and document your validation rules. Use standard command-line utilities and explain your pipeline.

6. Section C — Python Programming (25 Marks)

Maximum Marks: 25

Answer all five questions. Each question carries 5 marks.

Question 13. Functions, Collections and Data Validation [5 marks]

> Write a Python program that accepts student records and calculates totals and percentages, class average, highest and lowest scores, number of students above the class average, and a ranking ordered by total marks with a documented tie-breaking rule.
>
> **Requirements:** Use appropriate lists, dictionaries, functions and loops. Validate subject marks, handle an empty dataset, and separate data collection, calculations and display logic.

Question 14. File Processing and Exception Handling [5 marks]

> Develop a Python program that reads a CSV file containing student IDs, names and marks in three subjects. Detect missing fields and invalid numerical values, calculate totals and averages, write valid records to a new CSV file, record invalid rows in an error report, and handle missing input files and file-access errors gracefully.
>
> **Requirements:** The original input file must remain unchanged. Use the standard library CSV tools.

Question 15. Object-Oriented Programming [5 marks]

> Design a Python class named BankAccount with an account holder's name, account number and current balance, together with deposit, withdrawal and balance inquiry operations.
>
> Demonstrate the class using at least three test cases, including an invalid transaction. Explain the difference between a class and an instance and why validation belongs in the account's operations.
>
> **Requirements:** Reject non-positive transaction amounts, prevent withdrawals exceeding the available balance, and encapsulate account state through suitable methods and attributes.

Question 16. Recursion, Iteration and Complexity [5 marks]

> Implement a program that calculates the greatest common divisor (GCD) of two non-negative integers using both an iterative algorithm and a recursive algorithm.
>
> Test normal and boundary inputs. Explain the Euclidean algorithm and compare the approaches in time complexity, auxiliary space and stack usage.
>
> **Requirements:** Use GCD(0, n) = n for positive n and document how GCD(0, 0) is handled.

Question 17. Automated Testing and Debugging [5 marks]

> Write a Python module containing a function that checks whether an integer is prime. Create a separate test module using Python's built-in unit test framework.
>
> Test negative integers, zero and one, two and other small primes, composite numbers, a relatively large prime, and invalid input types according to your documented function contract. Explain boundary-value testing and include the test command and actual output.

7. Section D — Integrated Mini-Project (15 Marks)

Question 18. Student Performance Analytics and Report Generator [15 marks]

Develop a small integrated student-performance management system using C, Linux/Bash and Python. The project must show that each technology has an appropriate role and that the components communicate through documented input/output files.

Part A — C Data-Entry Utility

Create a C program that accepts student ID, name and marks in three subjects. Validate the input and write valid records to a documented CSV format.

Part B — Linux/Bash Execution Pipeline

- Verify required files and directories.
- Compile the C program.
- Execute it with suitable sample input.
- Run the Python analysis program.
- Store logs and reports in designated directories.
- Return an appropriate status when a step fails.

Part C — Python Analytics Engine

- Read the generated CSV file.
- Calculate totals, averages, rankings and subject-wise statistics.
- Identify students who require additional academic support using a documented threshold.
- Write a summary report and a ranked CSV file.
- Handle malformed records without silently corrupting valid data.

Part D — Documentation and Verification

- Provide a README with setup and execution instructions.
- Provide the project directory tree.
- Include at least five meaningful test cases.
- Provide genuine evidence of a successful end-to-end run.
- Discuss design choices, limitations and possible improvements.

Minimum Technical Standards

1. The system must run in a Linux environment using documented commands.
2. The C source must compile without warnings under the chosen warning flags.
3. Python must use the standard library unless an additional dependency is justified.
4. Input validation and error handling must be demonstrated.
5. The project must not contain real student personal data.
6. The components must have distinct and documented responsibilities.

> **Important:** A collection of unrelated C, Bash and Python files in one directory does not satisfy the integration requirement.

8. Submission Structure and Verification Checklist

Submit one compressed archive using the naming convention **Final_Assignment_RollNumber.zip**. The following directory structure is recommended:

```
Final_Assignment_RollNumber/
├── README.md
├── c_programs/
│   ├── q01_pointers.c
│   ├── q02_students.c
│   ├── q03_linked_list.c
│   ├── q04_inventory.c
│   ├── q05_bitwise.c
│   └── q07_palindrome.c
├── linux/
│   ├── q10_run_tests.sh
│   ├── q12_process_data.sh
│   └── diagnostics.md
├── python_programs/
│   ├── q13_students.py
│   ├── q14_csv_processing.py
│   ├── q15_bank_account.py
│   ├── q16_gcd.py
│   ├── q17_prime.py
│   └── test_prime.py
├── integrated_project/
│   ├── src/
│   ├── scripts/
│   ├── data/
│   └── reports/
├── tests/
│   ├── test_cases.md
│   └── expected_results.md
└── evidence/
    ├── compilation/
    ├── linux/
    ├── python_tests/
    └── integration/
```

Students may adjust filenames where necessary, but every numbered question must be clearly identifiable. Do not include compiled binaries, virtual environments or generated caches unless specifically requested.

Final Verification Checklist

- ☐ All required C source files compile and execute.
- ☐ All Bash scripts have been tested on Linux.
- ☐ Python programs and automated tests execute successfully.
- ☐ The integrated project runs from the documented instructions.
- ☐ README contains prerequisites, commands, data formats and limitations.
- ☐ Evidence contains genuine terminal output or screenshots.
- ☐ Test cases include inputs, expected results and actual results.
- ☐ The archive contains no secrets, private keys or real student records.

9. Assessment Rubric and Grading Guidelines

Component	Marks	Assessment Focus
C Programming	35	Correctness, pointers, data structures, memory safety, algorithms, file handling and debugging
Linux and Bash	25	Commands, permissions, process handling, scripting, diagnostics and data processing
Python Programming	25	Functions, collections, OOP, exceptions, algorithms and automated tests
Integrated Project	15	Integration, reliability, documentation and end-to-end execution
TOTAL	**100**	**Overall first-year practical competence**

Overall Evaluation Criteria

Criterion	Weight	Full-Credit Standard
Correctness	30%	Correct results, including boundary cases
Implementation and Design	25%	Appropriate language features, modularity and efficient algorithms
Error Handling and Reliability	15%	Invalid input, file errors and failure conditions handled safely
Testing and Verification	15%	Meaningful tests with reproducible evidence
Documentation and Code Quality	10%	Readable code, useful comments and clear instructions
Explanation and Originality	5%	Independent understanding and ability to justify decisions

The rubric percentages apply to the overall 100-mark assessment. Individual question marks remain the primary allocation; assessors may use the rubric to guide the distribution of marks within each question.

Proposed Performance Bands

Marks	Interpretation
90–100	A
75–89	B
60–74	C
50–59	D
Below 50	E

10. Academic Integrity and Technical Rules

1. **Individual work:** Every student must develop and understand their own submission.
2. **Permitted references:** Official documentation and course notes may be consulted. Disclose substantial external code or AI assistance in accordance with institutional policy.
3. **No fabricated evidence:** Logs, screenshots, benchmarks and test outputs must reflect actual execution.
4. **Safe programming:** Buffer overflows, invalid pointer use, memory leaks and uncontrolled file operations may lose correctness and reliability marks.
5. **Reproducibility:** Faculty must be able to follow the README and reproduce the main results.
6. **Code explanation:** Students must explain submitted functions, scripts, algorithms and design choices during viva voce.

7. **Environment:** Use a supported Linux distribution, GCC, Bash and Python 3. Record relevant versions in README.md.
8. **Late submission:** Apply only the institution's published late-submission policy.

11. Suggested 16-Day Work Plan

Period	Focus	Suggested Work
Days 1–3	C foundations and memory	Questions 1–3: pointers, structures, linked lists and memory safety
Days 4–5	C files and algorithms	Questions 4–7: persistent records, bitwise operations, debugging and strings
Days 6–8	Linux and Bash	Questions 8–12: permissions, processes, automation, diagnostics and text processing
Days 9–11	Python	Questions 13–17: data processing, OOP, recursion, exceptions and unit tests
Days 12–15	Integrated project	Implement the complete pipeline, test failure cases and prepare reports
Day 16	Final verification	Run tests from a clean environment, check the archive and rehearse viva

12. Viva Voce Question Bank

The viva should verify understanding rather than reward memorisation. Faculty may select questions from the following bank and may ask the student to modify a program live.

1. Why can dereferencing a freed pointer produce undefined behaviour?
2. When would a linked list be preferable to an array?
3. What is the difference between stack and heap allocation?
4. What is the difference between file permissions and file ownership?
5. How does a Bash pipeline pass data between processes?
6. Why should scripts check the exit status of a command?
7. What is the difference between a Python list and a dictionary?
8. How should a program handle malformed CSV data?
9. When is recursion less suitable than iteration?
10. How do unit tests improve confidence in a program?
11. How does your integrated project detect and report a failed stage?
12. What would you change if the dataset grew from 20 records to one million?

13. Student Declaration

I declare that this assignment is my own work except where sources or assistance have been acknowledged. The code, test results, screenshots and reports submitted by me are genuine and were produced or verified by me. I understand that I may be asked to explain or demonstrate any part of this submission.

Declaration Field	Student Entry
Student Name	
Roll / Registration Number	
Date	
Signature	

14. Faculty Assessment Record

Assessment Component	Maximum Marks	Marks Awarded	Remarks
Section A — C Programming	35		
Section B — Linux and Bash	25		
Section C — Python Programming	25		
Section D — Integrated Project	15		
TOTAL	**100**		

Faculty Record	Details
Evaluator Name	
Viva Marks / Notes (if separately recorded)	
Evaluator Comments	
Strengths	
Areas for Improvement	
Viva Voce Observations	

15. Final Evaluation and Approval

To be completed by the evaluator and the authorized officers after evaluation and viva voce.

Final Evaluation	Details
Total Marks Awarded (out of 100)	
Evaluator Signature	
Date	
Reviewer Name	
Reviewer Signature	
Date	
Authorized Approval (Name / Designation)	
Authorized Signature	
Date	

OFFICIAL DEPARTMENT SEAL / STAMP

— END OF ASSIGNMENT —